

SOFTWARE DOCUMENTATION

Patient Management System

Developer Guide

Document Type	Developer / Technical Guide
Version	1.0
Status	Released
Date	September 2026
Repository	https://github.com/cepdnaclk/e22-2yp-co2060-Patient-Management-System
Prepared by	Department of Computer Engineering, University of Peradeniya

Revision History

Version	Date	Author	Description
1.0	Sep 2026	Development Team	Initial release of the developer guide

Contents

1	Project Overview	4
2	Architecture	5
2.1	Billing Subsystem Architecture	5
3	Technology Stack	7
3.1	Backend	7
3.2	Frontend	7
4	Prerequisites	8
5	Environment Setup	9
5.1	Clone the Repository	9
5.2	Create the PostgreSQL Database	9
5.3	Set Environment Variables	9
5.3.1	Windows (PowerShell)	9
5.3.2	Linux / macOS	9
6	Backend Setup (Spring Boot)	10
6.1	Navigate to Backend Directory	10
6.2	Build and Run	10
6.2.1	Using Maven Wrapper (Recommended)	10
6.2.2	Using Maven Directly	10
6.3	Verify Startup	10
6.4	Build JAR for Production	10
6.5	Docker Build	10
7	Frontend Setup (React + Vite)	11
7.1	Navigate to Frontend Directory	11
7.2	Install Dependencies	11
7.3	Configure Environment	11
7.4	Start Development Server	11
7.5	Build for Production	11
8	Project Structure	12
8.1	Overall Layout	12
8.2	Backend Module Pattern	13
9	Authentication & Security	14
9.1	Authentication Flow	14
9.2	JWT Token Configuration	14
9.3	Password Security	14
9.4	Google OAuth Flow	14
9.5	Security Headers	14
9.6	CORS Configuration	15
9.7	Rate Limiting	15
10	API Reference	16
10.1	Authentication	16
10.2	Patients	16
10.3	Doctors	16
10.4	Appointments	16
10.5	Medical Records	17

10.6 Invoices (/api/invoices)	17
10.7 Pending Bill Items (/api/billing/pending-items)	18
10.8 Users (Admin Only)	18
10.9 Audit Logs (Admin Only)	18
10.10 File Operations	19
11 Database Schema	20
11.1 Tables	20
11.2 Key Relationships	20
11.3 Audit Columns	20
11.4 Database Migrations	20
12 Real-Time Notifications (WebSocket)	22
12.1 Configuration	22
12.2 Frontend Client	22
12.3 Backend	22
13 Role-Based Access Control	23
13.1 Role Definitions	23
13.2 Endpoint Authorization	23
13.3 Frontend Route Protection	24
13.4 Dashboard → Role Mapping	24
14 Frontend Architecture	25
14.1 State Management	25
14.2 API Client	25
14.3 Routing	25
14.4 Navigation	25
15 Backend Module Architecture	26
15.1 Module Overview	26
15.2 Configuration Classes	26
15.3 Exception Handling	26
15.4 Billing Domain Architecture & Lifecycle	26
15.4.1 Staged Staging Queue (PendingBillItem)	27
15.4.2 Consolidation & Atomic Invoicing (BillingService.createInvoice)	27
15.4.3 Payment Processing & State Machine (BillingService.recordPayment)	27
16 Deployment	28
16.1 Frontend → Vercel	28
16.2 Backend → Docker / Render	28
16.2.1 Docker	28
16.2.2 Render	28
16.2.3 Railway	28
16.3 CORS for Production	28
17 Testing	30
17.1 Backend Tests	30
17.2 Frontend Linting	30
17.3 Manual API Testing	30
18 Environment Variables Reference	31
18.1 Backend Environment Variables	31
18.2 Frontend Environment Variables	31
19 Coding Conventions	32
19.1 Backend (Java)	32

19.2 Frontend (React)	32
20 Troubleshooting (Developer)	33
20.1 Common Issues	33
20.2 Useful Debug Commands	33
21 Contributing	34
21.1 Workflow	34
21.2 Adding a New Backend Module	34
21.3 Adding a New Frontend Dashboard	34

1 Project Overview

The **Patient Management System (PMS)** is a full-stack web application for managing patient records, appointments, medical history, prescriptions, lab results, and billing in healthcare facilities. It features role-based access for 10 distinct user roles, JWT-based authentication with refresh tokens, Google OAuth integration, and real-time WebSocket notifications.

Repository: github.com/cepdnaclk/e22-2yp-co2060-Patient-Management-System

2 Architecture

The system follows a conventional three-tier architecture: a React single-page application communicates with a Spring Boot REST / WebSocket backend, which persists data in PostgreSQL.

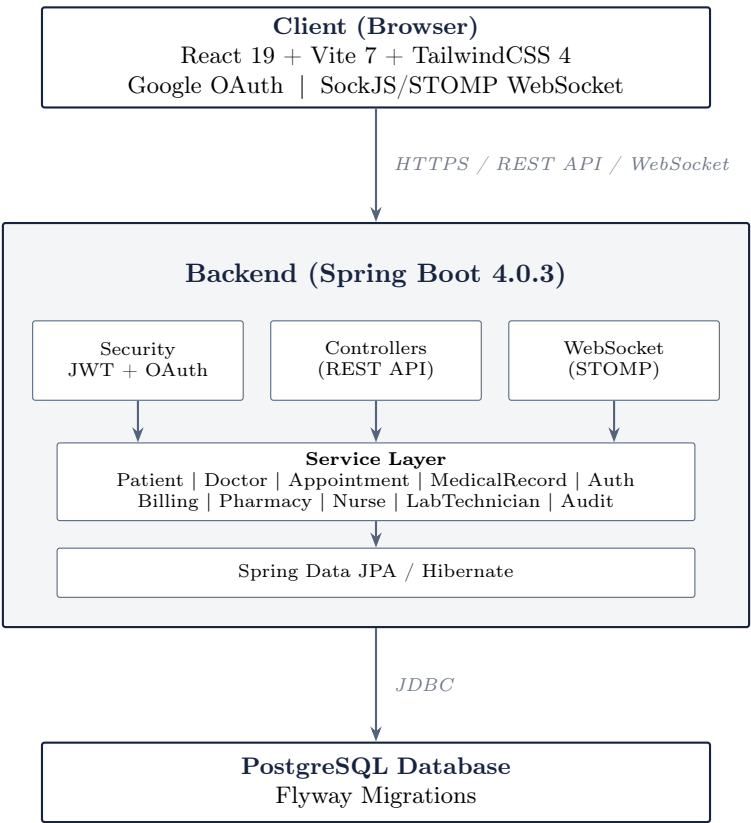


Figure 2.1: High-level system architecture

2.1 Billing Subsystem Architecture

The billing subsystem implements a **Decentralized Staging → Centralized Aggregation Pipeline** designed to decouple clinical operations from financial consolidation.

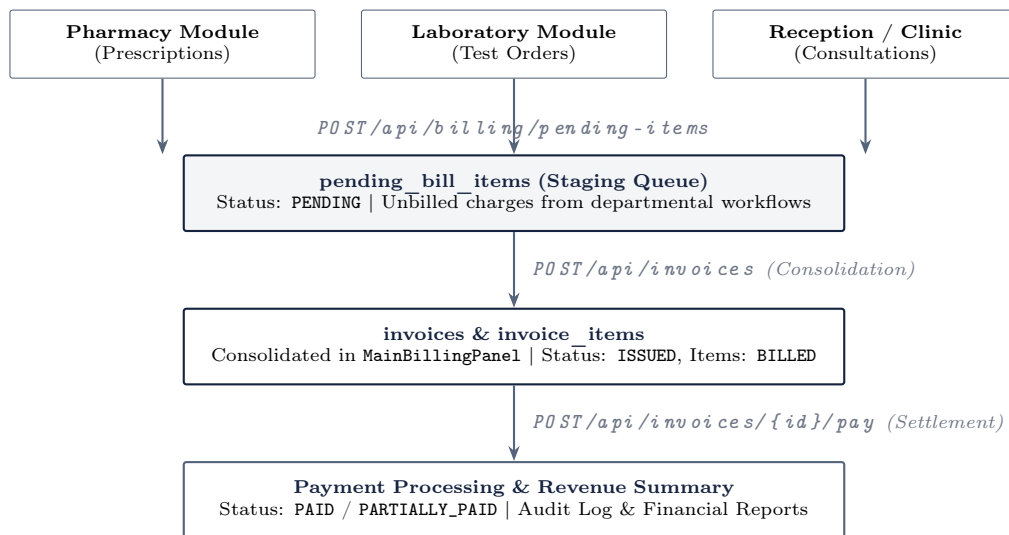


Figure 2.2: Decentralized billing staging and settlement pipeline

3 Technology Stack

3.1 Backend

Technology	Version	Purpose
Java	21	Programming language
Spring Boot	4.0.3	Application framework
Spring Security	—	Authentication & authorization
Spring Data JPA	—	Database ORM layer
Spring WebSocket	—	Real-time notifications (STOMP)
Hibernate	—	JPA implementation
PostgreSQL	—	Relational database
Flyway	—	Database migration management
JWT	0.12.6	JWT token generation/validation
Google API Client	2.6.0	Google OAuth token verification
Lombok	—	Boilerplate code reduction
Maven	—	Build tool and dependency management

3.2 Frontend

Technology	Version	Purpose
React	19.2	UI component library
Vite	7.3	Build tool and dev server
TailwindCSS	4.1	Utility-first CSS framework
React Router	7.13	Client-side routing
Axios	1.13	HTTP client
@react-oauth/google	0.13	Google Sign-In integration
@stomp/stompjs	7.3	WebSocket STOMP client
sockjs-client	1.6	WebSocket fallback
Lucide React	1.14	Icon library
qrcode.react	4.2	QR code generation

4 Prerequisites

Ensure the following are installed before setting up the project:

Tool	Minimum Version	Download
Java JDK	21	adoptium.net
Node.js	18 LTS	nodejs.org
npm	9+	Comes with Node.js
PostgreSQL	14+	postgresql.org
Git	2.30+	git-scm.com
Maven	3.9+ (or use mvnw)	Bundled via Maven Wrapper

5 Environment Setup

5.1 Clone the Repository

```
git clone https://github.com/cepdnaclk/e22-2yp-co2060-Patient-Management-System.git
cd e22-co2060-Patient-Management-System
```

5.2 Create the PostgreSQL Database

```
-- Connect to PostgreSQL and run:
CREATE DATABASE pms;
```

5.3 Set Environment Variables

5.3.1 Windows (PowerShell)

```
$env:JWT_SECRET = "your-super-secret-key-make-it-long-and-random-123"
$env:PGPASSWORD = "your-postgres-password"
$env:PGUSER = "postgres"
$env:PGHOST = "localhost"
$env:PGPORT = "5432"
$env:PGDATABASE = "pms"
```

5.3.2 Linux / macOS

```
export JWT_SECRET="your-super-secret-key-make-it-long-and-random-123"
export PGPASSWORD="your-postgres-password"
export PGUSER="postgres"
export PGHOST="localhost"
export PGPORT="5432"
export PGDATABASE="pms"
```

Note

See Chapter 18 for the full environment variables reference.

6 Backend Setup (Spring Boot)

6.1 Navigate to Backend Directory

```
cd code/backend
```

6.2 Build and Run

6.2.1 Using Maven Wrapper (Recommended)

```
# Windows
.\mvnw.cmd spring-boot:run

# Linux/macOS
./mvnw spring-boot:run
```

6.2.2 Using Maven Directly

```
mvn spring-boot:run
```

6.3 Verify Startup

- Check the console for: **Started BackendApplication**
- The API is available at: <http://localhost:8082>
- A **Super Admin** account is automatically seeded on first run:
 - **Email:** `admin@pms.local` (configurable via `PMS_SUPER_ADMIN_EMAIL`)
 - **Password:** `ChangeMe123!` (configurable via `PMS_SUPER_ADMIN_PASSWORD`)

6.4 Build JAR for Production

```
.\mvnw.cmd clean package -DskipTests
# Output: target/backend-0.0.1-SNAPSHOT.jar

java -jar target/backend-0.0.1-SNAPSHOT.jar
```

6.5 Docker Build

```
docker build -t pms-backend .
docker run -p 8082:8082 \
  -e JWT_SECRET="your-secret" \
  -e PGHOST="host.docker.internal" \
  -e PGPASSWORD="your-password" \
  pms-backend
```

7 Frontend Setup (React + Vite)

7.1 Navigate to Frontend Directory

```
cd code/frontend
```

7.2 Install Dependencies

```
npm install
```

7.3 Configure Environment

Copy the example environment file and edit:

```
cp .env.example .env
```

Edit `.env`:

```
VITE_API_URL=http://localhost:8082  
VITE_GOOGLE_CLIENT_ID=your-google-client-id.apps.googleusercontent.com
```

7.4 Start Development Server

```
npm run dev
```

The app runs at `http://localhost:5173` by default.

7.5 Build for Production

```
npm run build
```

Output is generated in the `dist/` directory.

8 Project Structure

8.1 Overall Layout

```
e22-co2060-Patient-Management-System/
|-- code/
|   |-- backend/                                # Spring Boot application
|   |   |-- src/main/java/com/pms/backend/
|   |   |   |-- BackendApplication.java
|   |   |   |-- admin/                        # Admin management module
|   |   |   |-- appointment/                 # Appointment scheduling
|   |   |   |-- audit/                       # Audit logging
|   |   |   |-- auth/                       # Authentication (JWT, OAuth)
|   |   |   |-- billing/                     # Billing domain (Invoices, Items, Pending Staging)
|   |   |   |   |-- controller/              # InvoiceController, PendingBillItemController
|   |   |   |   |-- dto/                    # CreateInvoiceRequest, CreatePendingItemRequest, PendingBillItemDto
|   |   |   |   |-- entity/                 # Invoice, InvoiceItem, PendingBillItem
|   |   |   |   |-- repository/             # InvoiceRepository, PendingBillItemRepository
|   |   |   |   |-- service/                # BillingService (aggregation & payments)
|   |   |   |   |-- common/                 # Shared exceptions & utilities
|   |   |   |   |-- config/                 # Security, CORS, WebSocket config
|   |   |   |   |-- doctor/                 # Doctor profiles & management
|   |   |   |   |-- file/                   # File upload handling
|   |   |   |   |-- management/             # Management module
|   |   |   |   |-- medicalrecord/          # Medical records (CRUD)
|   |   |   |   |-- notification/           # Real-time notifications
|   |   |   |   |-- nurse/                  # Nurse operations
|   |   |   |   |-- patient/                # Patient profiles
|   |   |   |   |-- pharmacy/               # Pharmacy module
|   |   |   |   |-- profilechange/          # Profile change requests
|   |   |   |   |-- role/                   # Role enum definition
|   |   |   |   |-- user/                   # User accounts & management
|   |   |-- src/main/resources/
|   |   |   |-- application.properties
|   |   |   |-- application-prod.properties
|   |   |   |-- db/migration/               # Flyway SQL migrations
|   |   |-- Dockerfile
|   |   |-- Procfile                        # Heroku deployment
|   |   |-- pom.xml                         # Maven dependencies
|   |-- frontend/                           # React application
|   |   |-- src/
|   |   |   |-- App.jsx                     # Root component & routes
|   |   |   |-- main.jsx                    # Entry point
|   |   |   |-- index.css                   # Global styles
|   |   |   |-- components/                 # Shared UI components
|   |   |   |   |-- Navbar.jsx
|   |   |   |   |-- NavbarLanding.jsx
|   |   |   |   |-- NotificationBell.jsx
|   |   |   |   |-- Sidebar.jsx
|   |   |   |   |-- AmbientOrbs.jsx
|   |   |   |   |-- ui/                     # Reusable UI primitives
|   |   |   |-- features/                   # Feature modules
|   |   |   |   |-- auth/                   # Login, Signup, AuthContext
|   |   |   |   |-- dashboard/              # Role-specific dashboards
|   |   |   |   |-- notifications/          # Notification components
|   |   |   |   |-- patients/               # Patient CRUD components
|   |   |   |   |-- theme/                  # Theme utilities
|   |   |   |-- layouts/                    # Layout wrappers
|   |   |   |   |-- DashboardLayout.jsx
|   |   |-- pages/                          # Public pages
|   |   |   |-- Home.jsx
|   |   |   |-- AboutUs.jsx
|   |   |   |-- ContactUs.jsx
|   |   |   |-- FAQ.jsx
|   |   |-- services/                       # API service modules
|   |   |   |-- axiosClient.js              # Configured Axios instance
|   |   |   |-- authService.js              # Auth token management
|   |   |   |-- patientDashboardService.js
|   |   |   |-- doctorDashboardService.js
|   |   |   |-- nurseDashboardService.js
|   |   |   |-- receptionistService.js
|   |   |   |-- pharmacyService.js
|   |   |   |-- labTechnicianService.js
```

```
|
|
|      |-- billingService.js
|      |-- adminService.js
|      |-- managementService.js
|      |-- patientRecordService.js
|      |-- fileUploadService.js
|      |-- profileChangeService.js
|      |-- .env.example          # Environment template
|      |-- vite.config.js
|      |-- tailwind.config.js
|      |-- vercel.json           # Vercel deployment config
|      '-- package.json
|
|-- docs/                       # Documentation
|-- README.md
'-- package.json               # Root workspace (optional)
```

8.2 Backend Module Pattern

Each backend module follows a consistent **layered architecture**:

```
module/
|-- entity/      # JPA entity (database table mapping)
|-- repository/  # Spring Data JPA repository interface
|-- dto/         # Data Transfer Objects (request/response shapes)
|-- service/     # Business logic
'-- controller/  # REST API endpoints
```

9 Authentication & Security

9.1 Authentication Flow



Figure 9.1: Initial login exchange

Subsequent requests attach the access token as a bearer header:

```
Authorization: Bearer <access_token>
```

When the access token expires, the client silently exchanges the refresh token for a new pair:

```
POST /api/auth/refresh { refreshToken }
--> 200 OK { accessToken, refreshToken }
```

9.2 JWT Token Configuration

Parameter	Value
Access Token Expiry	15 minutes (900,000 ms)
Refresh Token Expiry	7 days (604,800,000 ms)
Algorithm	HMAC-SHA (via JJWT library)
Storage (Client)	localStorage (pms_token, pms_refresh_token)

9.3 Password Security

- **Algorithm:** BCrypt with strength factor 12
- **Account Lockout:** 5 failed login attempts → 15-minute lockout

9.4 Google OAuth Flow

1. Frontend uses `@react-oauth/google` to get a Google ID token.
2. Token is sent to `POST /api/auth/google`.
3. Backend verifies the token using the Google API Client library.
4. If valid, creates/retrieves the user and returns JWT tokens.

9.5 Security Headers

The backend applies the following security headers:

- **X-Frame-Options:** DENY — Prevents clickjacking

- X-Content-Type-Options: nosniff — Prevents MIME sniffing
- Referrer-Policy: strict-origin-when-cross-origin
- Strict-Transport-Security: max-age=31536000; includeSubDomains — Forces HTTPS

9.6 CORS Configuration

Allowed origins (configurable via `FRONTEND_URL` env var):

```
http://localhost:5173
http://localhost:5174
http://localhost:3000
${FRONTEND_URL} (default: https://e22-co2060-patient-management-syste-three.vercel.app)
```

9.7 Rate Limiting

Endpoint	Limit
Login	5 attempts per minute
Signup	3 attempts per hour

10 API Reference

Base URL: `http://localhost:8082/api`

10.1 Authentication

Method	Endpoint	Description	Auth
POST	<code>/auth/signup</code>	Register a new user	No
POST	<code>/auth/login</code>	Log in and receive JWT tokens	No
POST	<code>/auth/google</code>	Google OAuth sign-in	No
POST	<code>/auth/refresh</code>	Refresh access token	No

10.2 Patients

Method	Endpoint	Description	Auth
POST	<code>/patients</code>	Create patient profile	Yes
GET	<code>/patients</code>	List all patients	Yes
GET	<code>/patients/{id}</code>	Get patient by ID	Yes
GET	<code>/patients/user/{userId}</code>	Get patient by user ID	Yes
PUT	<code>/patients/{id}</code>	Update patient profile	Yes
DELETE	<code>/patients/{id}</code>	Delete patient	Yes

10.3 Doctors

Method	Endpoint	Description	Auth
POST	<code>/doctors</code>	Create doctor profile	Yes
GET	<code>/doctors</code>	List all doctors	Yes
GET	<code>/doctors/{id}</code>	Get doctor by ID	Yes
GET	<code>/doctors/specialization/{spec}</code>	Search by specialization	Yes
GET	<code>/doctors/available</code>	Get available doctors	Yes
PUT	<code>/doctors/{id}</code>	Update doctor profile	Yes
DELETE	<code>/doctors/{id}</code>	Delete doctor	Yes

10.4 Appointments

Method	Endpoint	Description	Auth
POST	<code>/appointments</code>	Create appointment	Yes
GET	<code>/appointments</code>	List all appointments	Yes
GET	<code>/appointments/{id}</code>	Get by ID	Yes

Method	Endpoint	Description	Auth
GET	/appointments/patient/{patientId}	Get patient's appointments	Yes
GET	/appointments/doctor/{doctorId}	Get doctor's appointments	Yes
GET	/appointments/status/{status}	Filter by status	Yes
PUT	/appointments/{id}	Update appointment	Yes
PUT	/appointments/{id}/cancel	Cancel appointment	Yes
DELETE	/appointments/{id}	Delete appointment	Yes

Appointment Statuses: SCHEDULED, COMPLETED, CANCELLED, NO_SHOW

10.5 Medical Records

Method	Endpoint	Description	Auth
POST	/medical-records	Create medical record	Yes
GET	/medical-records	List all records	Yes
GET	/medical-records/{id}	Get by ID	Yes
GET	/medical-records/patient/{patientId}	Get patient's records	Yes
GET	/medical-records/doctor/{doctorId}	Get doctor's records	Yes
GET	/medical-records/type/{recordType}	Filter by type	Yes
PUT	/medical-records/{id}	Update record	Yes
DELETE	/medical-records/{id}	Delete record	Yes

Record Types: DIAGNOSIS, PRESCRIPTION, LAB_RESULT, IMAGING

10.6 Invoices (/api/invoices)

Method	Endpoint	Description	Auth / Roles
POST	/invoices	Create consolidated invoice	ADMIN, SUPER_ADMIN, BILLING_STAFF, RECEPTIONIST
GET	/invoices	List invoices (?page=0&size=10)	ADMIN, SUPER_ADMIN, BILLING_STAFF, RECEPTIONIST
GET	/invoices/{id}	Get invoice details and line items	ADMIN, SUPER_ADMIN, BILLING_STAFF, DOCTOR, RECEPTIONIST, PATIENT
POST	/invoices/{id}/pay	Record payment (amount, method)	ADMIN, SUPER_ADMIN, BILLING_STAFF, RECEPTIONIST

Method	Endpoint	Description	Auth / Roles
GET	/invoices/patient/{patientId}	Get invoices for specific patient	ADMIN, SUPER_ADMIN, BILLING_STAFF, DOCTOR, PATIENT
GET	/invoices/summary	Get financial revenue summary	ADMIN, SUPER_ADMIN, BILLING_STAFF

Invoice Statuses: ISSUED, PARTIALLY_PAID, PAID, CANCELLED

Payment Methods: CASH, CARD, INSURANCE, BANK_TRANSFER

10.7 Pending Bill Items (/api/billing/pending-items)

Method	Endpoint	Description	Auth / Roles
POST	/billing/pending-items	Batch enqueue departmental charges	ADMIN, SUPER_ADMIN, PHARMACIST, LAB_TECHNICIAN, RECEPTIONIST
GET	/billing/pending-items/patients	List patient IDs with unbilled items	ADMIN, SUPER_ADMIN, RECEPTIONIST, BILLING_STAFF
GET	/billing/pending-items/patient/{id}	Get pending charges for patient	ADMIN, SUPER_ADMIN, RECEPTIONIST, BILLING_STAFF

Origin Departments: PHARMACY, LAB, RECEPTION, OTHER

Item Statuses: PENDING (staged), BILLED (consolidated into invoice)

10.8 Users (Admin Only)

Method	Endpoint	Description	Auth
GET	/users	List all users	ADMIN, SUPER_ADMIN, MANAGEMENT, DOCTOR
Various	/users/**	User management	ADMIN, SUPER_ADMIN

10.9 Audit Logs (Admin Only)

Method	Endpoint	Description	Auth
GET	/audit/**	View audit logs	ADMIN, SUPER_ADMIN

10.10 File Operations

Method	Endpoint	Description	Auth
Various	/files/**	File download/access	Public

11 Database Schema

11.1 Tables

The schema is auto-managed by Hibernate (`ddl-auto=update`) with Flyway migrations for structural changes.

Table	Description
<code>users</code>	User accounts (email, password hash, role, active status)
<code>roles</code>	Role definitions (via enum)
<code>patients</code>	Patient profiles (demographics, medical history, allergies)
<code>doctors</code>	Doctor profiles (specialization, license, availability, fees)
<code>appointments</code>	Scheduled appointments (patient-doctor, status, notes)
<code>medical_records</code>	Clinical records (diagnosis, prescription, lab results, imaging)
<code>invoices</code>	Consolidated billing invoices (patient, created_by, status, total, paid, tax, discount)
<code>invoice_items</code>	Itemized charges under an invoice (description, quantity, unit_price, total_price, item_type)
<code>pending_bill_items</code>	Inter-departmental charge staging queue (patient, department, status: PENDING/BILLED)

11.2 Key Relationships

- `users` 1 : 1 $\rightarrow$ `patients`
- `users` 1 : 1 $\rightarrow$ `doctors`
- `patients` 1 : N $\rightarrow$ `appointments`
- `doctors` 1 : N $\rightarrow$ `appointments`
- `patients` 1 : N $\rightarrow$ `medical_records`
- `doctors` 1 : N $\rightarrow$ `medical_records`
- `patients` 1 : N $\rightarrow$ `invoices` 1 : N $\rightarrow$ `invoice_items`
- `patients` 1 : N $\rightarrow$ `pending_bill_items`
- `users` (staff) 1 : N $\rightarrow$ `invoices` (created_by)

11.3 Audit Columns

All tables include:

- `created_at` — Record creation timestamp
- `updated_at` — Last modification timestamp

11.4 Database Migrations

Flyway migration files are located at:

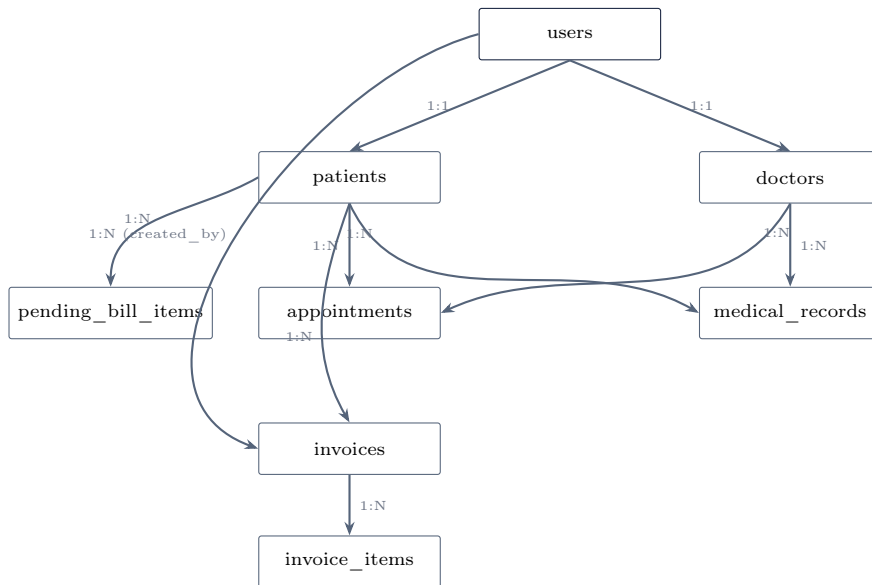


Figure 11.1: Entity relationships across core and billing domains

```
code/backend/src/main/resources/db/migration/
```

Migration naming convention: V{version}__{description}.sql

Key Migration Scripts:

- V1__init_schema.sql — Core users, roles, patients, doctors, appointments, medical records
- V2__add_refresh_tokens.sql — Refresh token authentication storage
- V3__add_billing.sql — Invoices, invoice items, and pending bill items staging schema
- V4__add_audit_logs.sql — Audit logging subsystem
- V5__add_notifications.sql — Real-time notification entity storage
- V14__add_nurse_workflow_tables.sql — Nurse vitals and medication orders
- V15__add_medicines_with_prices.sql — Pharmacy inventory pricing lookup

12 Real-Time Notifications (WebSocket)

12.1 Configuration

- **Endpoint:** /ws (SockJS fallback enabled)
- **Message Broker Prefix:** Configured via `WebSocketConfig.java`
- **Protocol:** STOMP over WebSocket/SockJS

12.2 Frontend Client

The frontend uses `@stomp/stompjs` with `sockjs-client` for WebSocket connectivity:

```
import { Client } from '@stomp/stompjs';
import SockJS from 'sockjs-client';

const client = new Client({
  websocketFactory: () => new SockJS(`${API_URL}/ws`),
  onConnect: () => {
    client.subscribe('/user/queue/notifications', (message) => {
      // Handle notification
    });
  },
});
client.activate();
```

12.3 Backend

WebSocket is configured in `WebSocketConfig.java` and CORS is enabled for the WebSocket handshake at `/ws/**`.

13 Role-Based Access Control

13.1 Role Definitions

Defined in `Role.java`:

Role	Description
SUPER_ADMIN	Full system access
ADMIN	User management, audit logs, reports
MANAGEMENT	Staff management (excludes admin accounts)
DOCTOR	Full clinical access
NURSE	Record vitals, assist doctors
RECEPTIONIST	Appointments, patient registration
PHARMACIST	Prescription management
LAB_TECHNICIAN	Lab results entry
BILLING_STAFF	Billing and payments
PATIENT	Own records only

13.2 Endpoint Authorization

Global URL filters are configured in `SecurityConfig.java`:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/ws/**").permitAll()
    .requestMatchers("/api/audit/**").hasAnyRole("ADMIN", "SUPER_ADMIN")
    .requestMatchers("/api/users/**").hasAnyRole("ADMIN", "SUPER_ADMIN", "MANAGEMENT", "DOCTOR")
    .requestMatchers("/api/files/**").permitAll()
    .anyRequest().authenticated()
)
```

Granular role authorization is enforced via `@PreAuthorize` on controller methods:

Controller Endpoint	Allowed Roles	Business Purpose
POST/api/billing/pending-items	ADMIN, SUPER_ADMIN, PHARMACIST, LAB_TECHNICIAN, RECEPTIONIST	Enqueue departmental service charges
GET/api/billing/pending-items/**	ADMIN, SUPER_ADMIN, RECEPTIONIST, BILLING_STAFF	Inspect staged charges to prepare bills
POST/api/invoices	ADMIN, SUPER_ADMIN, BILLING_STAFF, RECEPTIONIST	Create and issue consolidated invoices
POST/api/invoices/{id}/pay	ADMIN, SUPER_ADMIN, BILLING_STAFF, RECEPTIONIST	Record customer payments

Controller Endpoint	Allowed Roles	Business Purpose
GET/api/invoices/summary	ADMIN, SUPER_ADMIN, BILLING_STAFF	Access financial revenue statistics
GET/api/invoices/patient/{id}	ADMIN, SUPER_ADMIN, BILLING_STAFF, DOCTOR, PATIENT	Access personal or patient invoices

13.3 Frontend Route Protection

Routes are protected using the `ProtectedRoute` component:

```
<Route path="/dashboard/doctor"
  element={
    <ProtectedRoute allowedRoles={["DOCTOR", "ADMIN", "SUPER_ADMIN"]}>
      <DoctorDashboard />
    </ProtectedRoute>
  }
/>
```

13.4 Dashboard → Role Mapping

Route	Allowed Roles
/dashboard/doctor	DOCTOR, ADMIN, SUPER_ADMIN
/dashboard/nurse	NURSE, ADMIN, SUPER_ADMIN
/dashboard/patient	PATIENT
/dashboard/admin	ADMIN, SUPER_ADMIN, MANAGEMENT
/dashboard/management	MANAGEMENT
/dashboard/receptionist	RECEPTIONIST, ADMIN, SUPER_ADMIN
/dashboard/pharmacist	PHARMACIST, ADMIN, SUPER_ADMIN
/dashboard/labtechnician	LAB_TECHNICIAN, ADMIN, SUPER_ADMIN
/dashboard/billingstaff	BILLING_STAFF, ADMIN, SUPER_ADMIN

14 Frontend Architecture

14.1 State Management

- **AuthContext** (`features/auth/AuthContext.jsx`) — Global authentication state via React Context.
- **Component-Level State** — `useState` and `useEffect` hooks for local component state.
- **Token Storage** — `localStorage` keys:
 - `pms_token` — Access token
 - `pms_refresh_token` — Refresh token
 - `pms_user` — Serialized user object

14.2 API Client

The centralized Axios client (`services/axiosClient.js`) provides:

- **Base URL configuration** from the `VITE_API_URL` environment variable.
- **Request interceptor** — Automatically attaches the `Authorization: Bearer <token>` header.
- **Response interceptor** — Handles 401 errors by attempting automatic token refresh.
- **Token refresh flow** — When a 401 is received, the interceptor calls `/api/auth/refresh` with the stored refresh token and retries the original request.

14.3 Routing

Routing is defined in `App.jsx` using React Router v7:

- **Public Routes:** `/`, `/about`, `/contact`, `/faq`, `/signup`, `/login`
- **Protected Routes:** `/dashboard/*` (guarded by `ProtectedRoute`)
- **Error Routes:** `/unauthorized (403)`, `*` (404 catch-all)

14.4 Navigation

- **NavbarLanding** — Displayed on public/landing pages.
- **Navbar** — Displayed on authenticated dashboard pages (includes `NotificationBell`).
- **Sidebar** — Dashboard sidebar for navigation within the dashboard.

15 Backend Module Architecture

15.1 Module Overview

Module	Purpose
auth	Login, signup, Google OAuth, JWT token management
patient	Patient CRUD, profile management
doctor	Doctor CRUD, specialization search, availability
appointment	Appointment scheduling, status management
medicalrecord	Medical records (diagnosis, prescription, lab, imaging)
user	User account management
role	Role enum definition
admin	Administrative operations
management	Staff management operations
nurse	Nurse-specific operations
pharmacy	Prescription management
billing	Billing domain (invoices, items, pending staging, payments)
file	File upload/download handling
notification	Real-time notification service
audit	Audit log recording and retrieval
profilechange	Profile change request workflow
config	Security, CORS, WebSocket, DataSource configuration
common	Shared exceptions (<code>AppException</code> , <code>GlobalExceptionHandler</code>)

15.2 Configuration Classes

Class	Purpose
<code>SecurityConfig</code>	HTTP security filter chain, CORS, password encoder
<code>WebSocketConfig</code>	STOMP WebSocket endpoint and broker configuration
<code>DataSourceConfig</code>	Database connection pool configuration
<code>RateLimitingFilter</code>	Request rate limiting for login/signup
<code>UserDetailsServiceImpl</code>	Loads user details for Spring Security

15.3 Exception Handling

Global exception handling is managed by `GlobalExceptionHandler`:

- `AppException` — Custom application-level exceptions with HTTP status codes.
- All unhandled exceptions return structured JSON error responses.

15.4 Billing Domain Architecture & Lifecycle

The billing subsystem resolves multi-department charge synchronization through an event staging model:

15.4.1 Staged Staging Queue (PendingBillItem)

- Clinical modules (Pharmacy, Lab, Reception) do not directly manipulate finalized invoices. Instead, they write to `pending_bill_items` with initial status `PENDING`.
- Each item tracks origin department (`PHARMACY`, `LAB`, `RECEPTION`, `OTHER`), patient ID, description, quantity, unit price, and computed total.

15.4.2 Consolidation & Atomic Invoicing (BillingService.createInvoice)

When billing staff trigger invoice creation, the backend:

1. Generates a sequence invoice number: `INV-YYYY-XXXXX` (derived from database identifier sequence).
2. Creates an `Invoice` entity with status `ISSUED`, applying optional taxes, discounts, and payment terms.
3. Maps line items into `InvoiceItem` records attached to the invoice.
4. Automatically queries all `PENDING` items for that patient and transitions them to `BILLED` (`pendingBillItem.setStatus` preventing duplicate invoicing).
5. Records an asynchronous audit log entry (`CREATE_INVOICE`).

15.4.3 Payment Processing & State Machine (BillingService.recordPayment)

Invoices support incremental partial payments or lump-sum settlements:

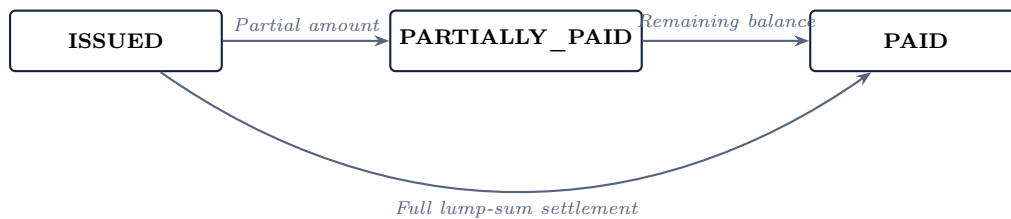


Figure 15.1: Invoice payment settlement state transitions

Payments update `paidAmount` and record transaction timestamps. Audit logs capture financial transactions (`RECORD_PAYMENT`) with exact amounts and remaining balance.

16 Deployment

16.1 Frontend → Vercel

1. Push code to GitHub.
2. Import the project in the [Vercel Dashboard](#).
3. Set **Root Directory** to `code/frontend`.
4. Set **Framework Preset** to `Vite`.
5. Configure environment variables:
 - `VITE_API_URL` = Backend URL (e.g., `https://e22-co2060-patient-management-system.onrender.com`)
 - `VITE_GOOGLE_CLIENT_ID` = Google OAuth client ID
6. Deploy. Future pushes to `main` auto-deploy.

SPA routing is handled by `vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

16.2 Backend → Docker / Render

16.2.1 Docker

```
cd code/backend
docker build -t pms-backend .
docker run -p 8082:8082 \
  -e JWT_SECRET="..." \
  -e PGHOST="..." \
  -e PGPASSWORD="..." \
  pms-backend
```

16.2.2 Render

A Procfile is provided:

```
web: java -jar target/backend-0.0.1-SNAPSHOT.jar
```

16.2.3 Railway

Database connection is configured via Railway's PostgreSQL environment variables (`PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`).

16.3 CORS for Production

Update the `FRONTEND_URL` environment variable on the backend to the deployed frontend URL:

FRONTEND_URL=<https://e22-co2060-patient-management-syste-three.vercel.app/>

17 Testing

17.1 Backend Tests

```
cd code/backend

# Run all tests
.\mvnw.cmd test

# Run specific test class
.\mvnw.cmd test -Dtest=PatientServiceTest
```

Test dependencies:

- spring-boot-starter-data-jpa-test
- spring-boot-starter-security-test
- spring-boot-starter-webmvc-test
- spring-boot-starter-security-oauth2-client-test

17.2 Frontend Linting

```
cd code/frontend
npm run lint
```

17.3 Manual API Testing

Use curl or Postman:

```
# Register a new user
curl -X POST http://localhost:8082/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "password": "SecurePass123!",
    "mobileNumber": "0712345678",
    "role": "PATIENT"
  }'

# Login
curl -X POST http://localhost:8082/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "john@example.com",
    "password": "SecurePass123!"
  }'

# Access protected endpoint (use token from login response)
curl -X GET http://localhost:8082/api/patients \
  -H "Authorization: Bearer <your-jwt-token>"
```

18 Environment Variables Reference

18.1 Backend Environment Variables

Variable	Default	Description
JWT_SECRET	(base64 string)	Secret key for JWT signing. Must be set in production.
PGHOST	localhost	PostgreSQL host
PGPORT	5432	PostgreSQL port
PGDATABASE	pms	PostgreSQL database name
PGUSER	postgres	PostgreSQL username
PGPASSWORD	—	PostgreSQL password. Must be set.
PORT	8082	Server HTTP port
FRONTEND_URL	https://e22-co2060-patient-management-systeme-three.vercel.app	CORS allowed origin
GOOGLE_TOKEN	(empty)	Google OAuth Client ID for backend verification
PMS_SUPER_ADMIN_EMAIL	admin@pms.local	Seeded super admin email
PMS_SUPER_ADMIN_PASSWORD	ChangeMe123!	Seeded super admin password
PMS_SUPER_ADMIN_FIRST_NAME	System	Super admin first name
PMS_SUPER_ADMIN_LAST_NAME	Admin	Super admin last name
PMS_SUPER_ADMIN_MOBILE	0000000000	Super admin mobile number
UPLOAD_DIR	uploads/lab-reports	File upload directory

18.2 Frontend Environment Variables

Variable	Description
VITE_API_URL	Backend API base URL (e.g., <code>https://e22-co2060-patient-management-system.onrender.com</code>)
VITE_GOOGLE_CLIENT_ID	Google OAuth Client ID for the frontend sign-in button

19 Coding Conventions

19.1 Backend (Java)

- **Package Structure:** `com.pms.backend.<module>.<layer>` (e.g., `com.pms.backend.patient.service`)
- **Naming:** PascalCase for classes, camelCase for methods and variables.
- **Annotations:** Use Lombok (`@Data`, `@Builder`, `@RequiredArgsConstructor`) to reduce boilerplate.
- **DTOs:** Always use DTOs for API request/response shapes. Never expose entities directly.
- **Validation:** Use `spring-boot-starter-validation` annotations (`@NotBlank`, `@Email`, etc.) on DTOs.
- **Security:** Use `@PreAuthorize` on controller methods for fine-grained role checks.

19.2 Frontend (React)

- **Components:** Functional components with hooks. No class components.
- **File Naming:** PascalCase for components (`.jsx`), camelCase for services (`.js`).
- **State:** React Context for global state (`AuthContext`), `useState`/`useEffect` for local state.
- **API Calls:** All API calls go through service files in `src/services/`.
- **Styling:** TailwindCSS utility classes. Custom CSS in `index.css` and `App.css`.

20 Troubleshooting (Developer)

20.1 Common Issues

Issue	Solution
JWT_SECRET not found	Set the JWT_SECRET environment variable before starting the backend.
Database connection refused	Ensure PostgreSQL is running and credentials are correct. Check PGHOST, PGPORT, PGUSER, PGPASSWORD.
CORS errors in browser	Ensure the frontend URL is in the backend's SecurityConfig.corsSource() allowed origins, or set via FRONTEND_URL.
users_role_check constraint error	The app auto-drops this constraint on startup. If it persists, manually run: ALTER TABLE users DROP CONSTRAINT IF EXISTS users_role_check;
Flyway migration conflict	If the schema has changed manually, set spring.flyway.baseline-on-migrate=true (already configured).
Port 8082 already in use	Kill the existing process or change the PORT environment variable.
Frontend VITE_API_URL not working	Environment variables must be prefixed with VITE_ for Vite to expose them. Restart the dev server after changing .env.
Google Sign-In not working	Ensure VITE_GOOGLE_CLIENT_ID (frontend) and GOOGLE_TOKEN (backend) are set to matching Google OAuth credentials.

20.2 Useful Debug Commands

```
# Check if backend is running
curl http://localhost:8082/api/auth/login -X POST -H "Content-Type: application/json" -d '{}' -v

# Check database connection
psql -h localhost -U postgres -d pms -c "SELECT 1;"

# View backend logs
.\mvnw.cmd spring-boot:run 2>&1 | tee backend.log
```

21 Contributing

21.1 Workflow

1. Fork the repository.
2. Create a feature branch: `git checkout -b feature/your-feature-name`
3. Make your changes following the coding conventions (Chapter 19).
4. Test your changes locally (both frontend and backend).
5. Commit with a descriptive message: `git commit -m "Add: patient search by name"`
6. Push to your fork: `git push origin feature/your-feature-name`
7. Open a Pull Request against the `main` branch.

21.2 Adding a New Backend Module

1. Create the module package: `com.pms.backend.<module_name>`
2. Create sub-packages: `entity`, `repository`, `dto`, `service`, `controller`
3. Define the JPA entity with `@Entity` and Lombok annotations.
4. Create the repository interface extending `JpaRepository`.
5. Create DTOs for API request/response.
6. Implement the service with business logic.
7. Create the REST controller with appropriate `@PreAuthorize` annotations.
8. Add a Flyway migration if schema changes are needed.

21.3 Adding a New Frontend Dashboard

1. Create the dashboard component in `src/features/dashboard/`.
2. Create a corresponding service file in `src/services/`.
3. Add the route in `App.jsx` wrapped with `<ProtectedRoute>`.
4. Add the role to `AuthContext` if needed.

Patient Management System — Developer Guide
Department of Computer Engineering, University of Peradeniya
Last Updated: September 2026